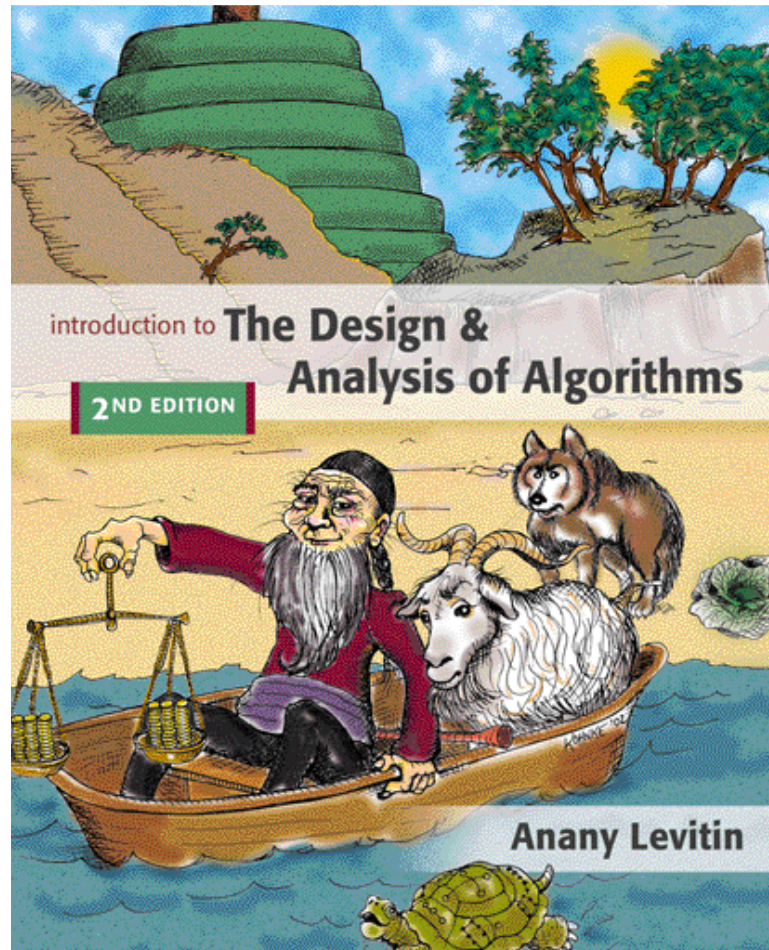


Analysis of Algorithms



Analysis of algorithms

- Issues:

- correctness
- time efficiency
- space efficiency
- optimality

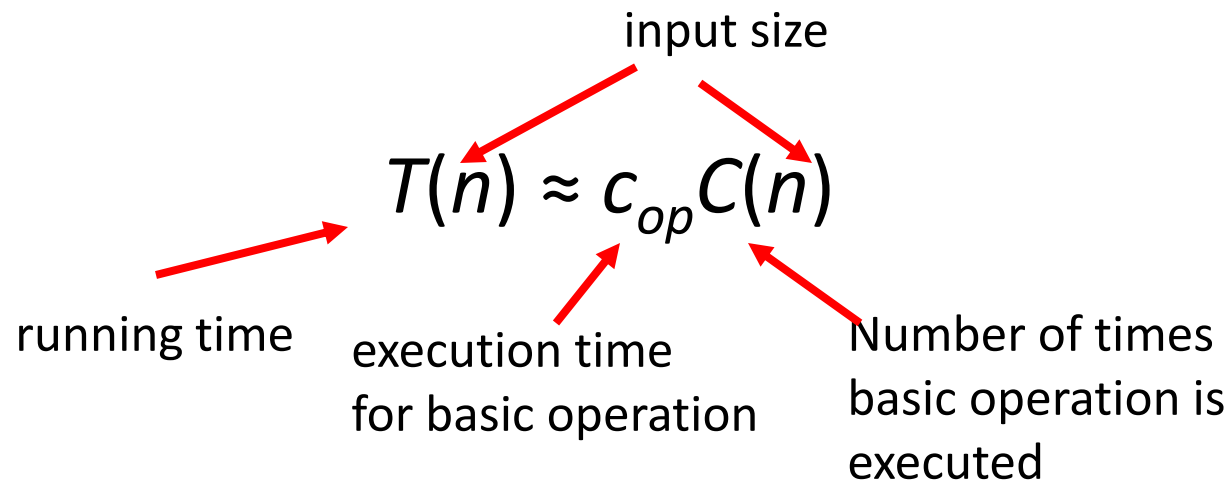
- Approaches:

- theoretical analysis
- empirical analysis

Theoretical analysis of time efficiency

Time efficiency is analyzed by determining the number of repetitions of the basic operation as a function of input size

- Basic operation: the operation that contributes most towards the running time of the algorithm



Input size and basic operation examples

<i>Problem</i>	<i>Input size measure</i>	<i>Basic operation</i>
Searching for key in a list of n items	Number of list's items, i.e. n	Key comparison
Multiplication of two matrices	Matrix dimensions or total number of elements	Multiplication of two numbers
Checking primality of a given integer n	n 'size = number of digits (in binary representation)	Division
Typical graph problem	#vertices and/or edges	Visiting a vertex or traversing an edge

Empirical analysis of time efficiency

- Select a specific (typical) sample of inputs
- Use physical unit of time (e.g., milliseconds)
or
Count actual number of basic operation's executions
- Analyze the empirical data

Best-case, average-case, worst-case

For some algorithms efficiency depends on form of input:

- Worst case: $C_{\text{worst}}(n)$ – maximum over inputs of size n
- Best case: $C_{\text{best}}(n)$ – minimum over inputs of size n
- Average case: $C_{\text{avg}}(n)$ – “average” over inputs of size n
 - Number of times the basic operation will be executed on typical input
 - NOT the average of worst and best case
 - Expected number of basic operations considered as a random variable under some assumption about the probability distribution of all possible inputs

Example: Sequential search

ALGORITHM *SequentialSearch*($A[0..n - 1]$, K)

//Searches for a given value in a given array by sequential search

//Input: An array $A[0..n - 1]$ and a search key K

//Output: The index of the first element of A that matches K

// or -1 if there are no matching elements

$i \leftarrow 0$

while $i < n$ **and** $A[i] \neq K$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1

- Worst case
- Best case
- Average case

Average Case of Sequential Search

Average case: Assumption

(a) the probability of a successful search is equal to p , $0 \leq p \leq 1$

(b) the probability of the first match occurring in the i^{th} position of the list is the same for every i

1 [success] the probability of the first match occurring in the i^{th} position of the list is p/n . the number of comparison is i .

2 [unsuccess] the number of comparison is n .

Therefore:

$$\begin{aligned} C_{\text{avg}}(n) &= [1.p/n + 2.p/n + \dots + i.p/n + \dots + n.p/n] + n.(1-p) \\ &= p/n[1+2+\dots+i+\dots+n] + n(1-p) \\ &= p/n \cdot n(n+1)/2 + n(1-p) = p(n+1)/2 + n(1-p) \end{aligned}$$

Puzzle



Puzzle

- Glove Selection: There are 22 gloves in a drawer: 5 pairs of red groves, 4 pairs of yellow, and 2 pairs of green. You select grove in the dark and can check them only after the selection has been made. What is the smallest number of groves you need to select to have at least one matching pair:
 - in the best case?
 - in the worst case?

Types of formulas for basic operation's count

- Exact formula

e.g., $C(n) = n(n-1)/2$

- Formula indicating order of growth with specific multiplicative constant

e.g., $C(n) \approx 0.5 n^2$

- Formula indicating order of growth with unknown multiplicative constant

e.g., $C(n) \approx cn^2$

Order of growth

- Most important: Order of growth within a constant multiple as $n \rightarrow \infty$
- Example:
 - How much faster will algorithm run on computer that is twice as fast?
 - How much longer does it take to solve problem of double input size?

Double Input Size

Example: Assuming that $C(n) = \frac{1}{2} n (n-1)$

How much longer will the algorithm run if we double its input size, $T(2n)$?

Values of some important functions as $n \rightarrow \infty$

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

Table 2.1 Values (some approximate) of several functions important for analysis of algorithms

Asymptotic order of growth

A way of comparing functions that ignores constant factors and small input sizes

- $O(g(n))$: class of functions $f(n)$ that grow no faster than $g(n)$
- $\Theta(g(n))$: class of functions $f(n)$ that grow at same rate as $g(n)$
- $\Omega(g(n))$: class of functions $f(n)$ that grow at least as fast as $g(n)$

Big-oh

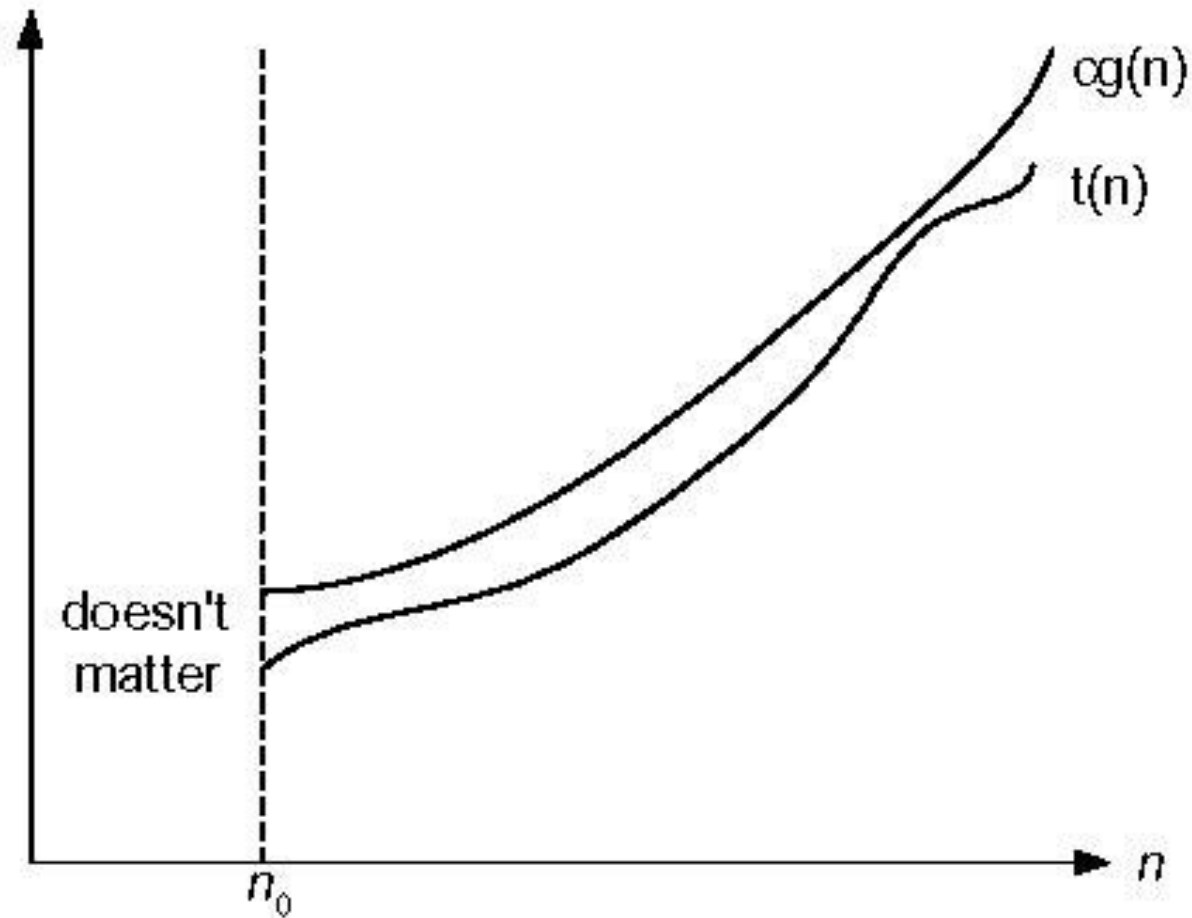


Figure 2.1 Big-oh notation: $t(n) \in O(g(n))$

A. Levitin, "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 2 ©2012 Pearson Education, Inc.
Upper Saddle River, NJ. All Rights Reserved.

Big-omega

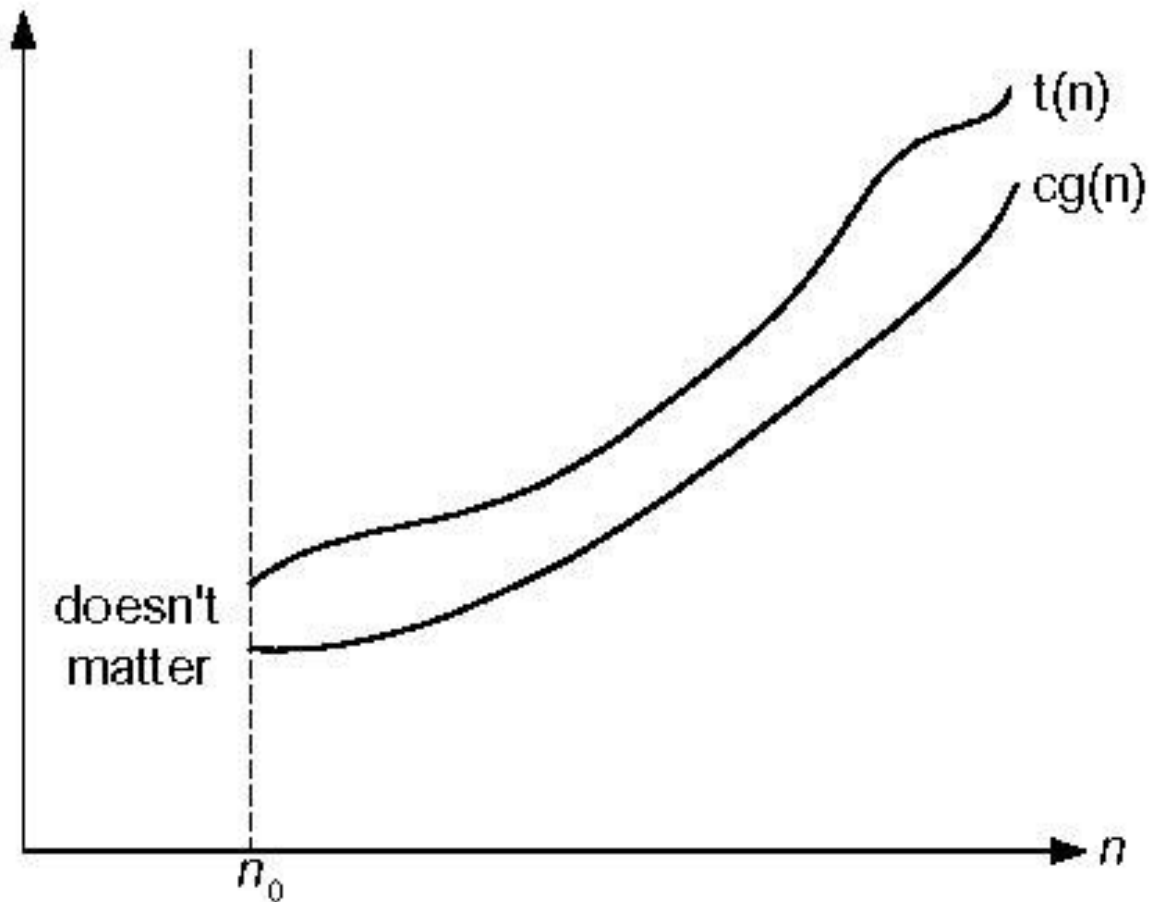


Fig. 2.2 Big-omega notation: $t(n) \in \Omega(g(n))$

Big-theta

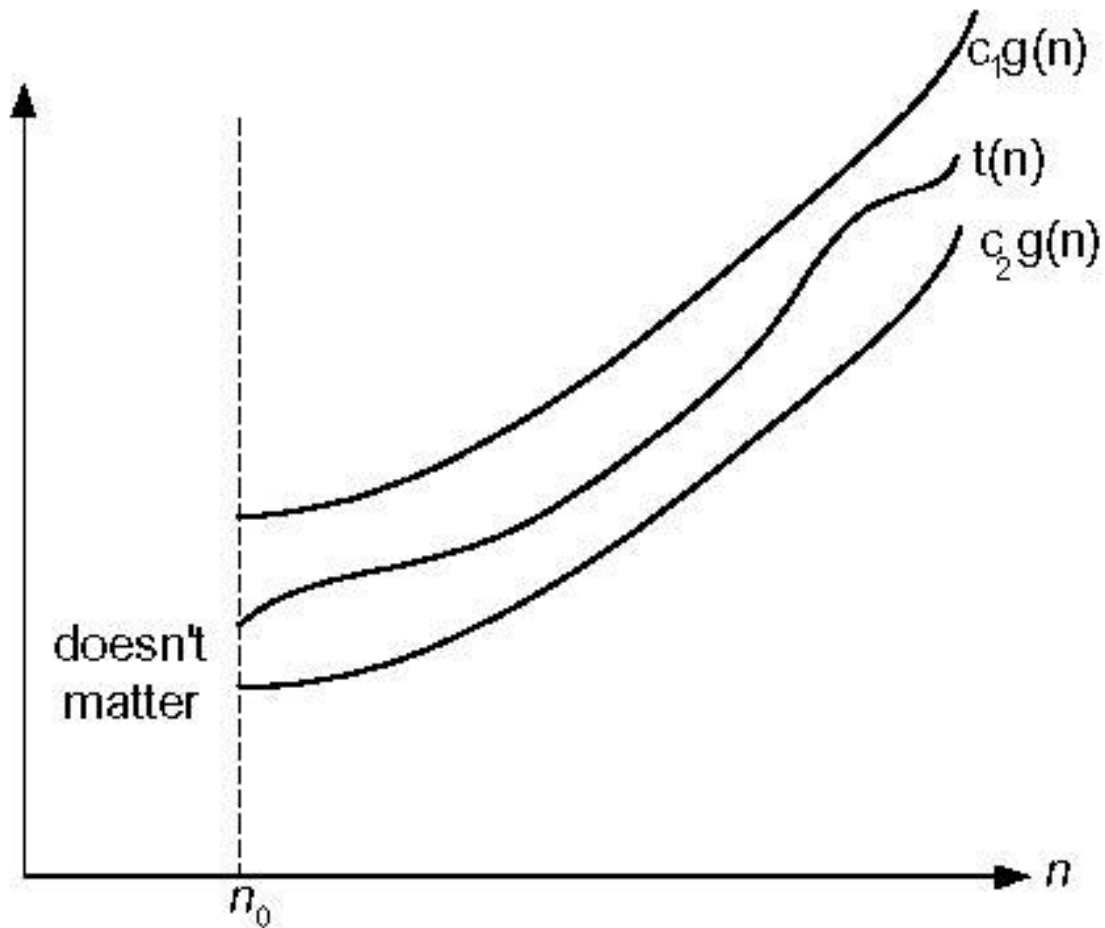


Figure 2.3 Big-theta notation: $t(n) \in \Theta(g(n))$

Properties of Logarithms

$$\log_a 1 = 0 \qquad \lg x \text{ is } \log_2 x$$

$$\log_a x = 1/(\log_x a)$$

$$\log_a a = 1$$

$$\log_a x^y = y \log_a x$$

$$\log_a xy = \log_a x + \log_a y$$

$$\log_a (x/y) = \log_a x - \log_a y$$

$$\log_a x = (\log_b x)/(\log_b a) = (\log_b x)(\log_a b)$$

$$a_{\log_b x} = x_{\log_b a}$$

Establishing order of growth using the definition

Definition: $f(n)$ is in $O(g(n))$ if order of growth of $f(n) \leq$ order of growth of $g(n)$ (within constant multiple),
i.e., there exist positive constant c and non-negative integer n_0 such that

$$f(n) \leq c g(n) \text{ for every } n \geq n_0$$

Examples:

- $10n$ is $O(n^2)$
- $5n+20$ is $O(n)$

Establishing order of growth using the definition

Definition: $f(n)$ is in $\Omega(g(n))$ if order of growth of $f(n) \geq$ order of growth of $g(n)$ (within constant multiple),
i.e., there exist positive constant c and non-negative integer n_0 such that

$$f(n) \geq c g(n) \text{ for every } n \geq n_0$$

Examples:

- $10n$ is $\Omega(n)$
- $n^2+5n+20$ is $\Omega(n^2)$
- $n^2+5n+20$ is $\Omega(n)$

Establishing order of growth using the definition

Definition: $f(n)$ is in $\Theta(g(n))$ if order of growth of $f(n)$ = order of growth of $g(n)$ (within constant multiple),
i.e., there exist positive constants c_1 and c_2 and non-negative integer n_0 such that

$$c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for every } n \geq n_0$$

Examples:

- $10n$ is $\Theta(n)$
- $5n+20$ is $\Theta(n)$
- $n^2+5n+20$ is $\Theta(n^2)$

Some properties of asymptotic order of growth

- $f(n) \in O(f(n))$
- $f(n) \in O(g(n))$ iff $g(n) \in \Omega(f(n))$
- If $f(n) \in O(g(n))$ and $g(n) \in O(h(n))$, then $f(n) \in O(h(n))$

Note similarity with $a \leq b$

- If $f_1(n) \in O(g_1(n))$ and $f_2(n) \in O(g_2(n))$, then
$$f_1(n) + f_2(n) \in O(\max\{g_1(n), g_2(n)\})$$

$O(\quad)?$

for (i = 1 to n)

....

....

for (j=1 to n)

for (k=1 to n)

....

....

for (m = 1 to n)

....

....

Establishing order of growth using limits

$$\lim_{n \rightarrow \infty} T(n)/g(n) = \begin{cases} 0 & \text{order of growth of } T(n) < \text{order of growth of } g(n) \\ c > 0 & \text{order of growth of } T(n) = \text{order of growth of } g(n) \\ \infty & \text{order of growth of } T(n) > \text{order of growth of } g(n) \end{cases}$$

Examples:

• $10n$ vs. n^2

• $n(n+1)/2$ vs. n^2

L'Hôpital's rule and Stirling's formula

L'Hôpital's rule: If $\lim_{n \rightarrow \infty} f(n) = \lim_{n \rightarrow \infty} g(n) = \infty$ and the derivatives f', g' exist, then

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{f'(n)}{g'(n)}$$

Example: $\log n$ vs. n

Stirling's formula: $n! \approx (2\pi n)^{1/2} (n/e)^n$

Example: 2^n vs. $n!$

Orders of growth of some important functions

- All logarithmic functions $\log_a n$ belong to the same class $\Theta(\log n)$ no matter what the logarithm's base $a > 1$ is
- All polynomials of the same degree k belong to the same class: $a_k n^k + a_{k-1} n^{k-1} + \dots + a_0 \in \Theta(n^k)$
- Exponential functions a^n have different orders of growth for different a 's
- order $\log n < \text{order } n^\alpha \ (\alpha > 0) < \text{order } a^n < \text{order } n! < \text{order } n^n$

Basic asymptotic efficiency classes

1	constant
$\log n$	logarithmic
n	linear
$n \log n$	n -log- n or linearithmic
n^2	quadratic
n^3	cubic
2^n	exponential
$n!$	factorial

Time efficiency of nonrecursive algorithms

General Plan for Analysis

- Decide on parameter n indicating input size
- Identify algorithm's basic operation
- Determine worst, average, and best cases for input of size n
- Set up a sum for the number of times the basic operation is executed
- Simplify the sum using standard formulas and rules (see Appendix A)

Summation

$$\sum_{i=1}^n 1 = n$$

$$\sum_{i=1}^u ca_i = c \sum_{i=1}^u a_i$$

$$\sum_{i=k}^u a_i = \sum_{i=k}^m a_i + \sum_{i=m+1}^u a_i$$

$$\sum_{i=k}^u (a_i - a_{i-1}) = a_u - a_{k-1}$$

$$\sum_{i=1}^n i2^i = (n-1)2^{n+1} + 2$$

Useful summation formulas and rules

$$\sum_{l \leq i \leq u} 1 = 1 + 1 + \cdots + 1 = u - l + 1$$

$$\text{In particular, } \sum_{l \leq i \leq u} 1 = n - 1 + 1 = n \in \Theta(n)$$

$$\sum_{1 \leq i \leq n} i = 1 + 2 + \cdots + n = n(n+1)/2 \approx n^2/2 \in \Theta(n^2)$$

$$\sum_{1 \leq i \leq n} i^2 = 1^2 + 2^2 + \cdots + n^2 = n(n+1)(2n+1)/6 \approx n^3/3 \in \Theta(n^3)$$

$$\sum_{0 \leq i \leq n} a^i = 1 + a + \cdots + a^n = (a^{n+1} - 1)/(a - 1) \text{ for any } a \neq 1$$

$$\text{In particular, } \sum_{0 \leq i \leq n} 2^i = 2^0 + 2^1 + \cdots + 2^n = 2^{n+1} - 1 \in \Theta(2^n)$$

$$\sum (a_i \pm b_i) = \sum a_i \pm \sum b_i \quad \sum c a_i = c \sum a_i \quad \sum_{l \leq i \leq u} a_i = \sum_{l \leq i \leq m} a_i + \sum_{m+1 \leq i \leq u} a_i$$

Principle of Counting

- Arrangement n^r (with Repetition)

- Permutation without repetition

$$P(n) = n!$$

$$P(n,r) = n! / [n-r]!$$

- Combination without repetition

$$C(n,r) = n! / [r! (n-r)!]$$

- Combination with repetition

$$C(n+r-1, r)$$

Mathematical Induction

2 Steps:

1) Basic step: is $s(1)$ true?

2) Inductive step:

if $s(k)$ is true, is $s(k+1)$ true?

- Domino

Homework Assignment

- Exercise 2.1 Page 76
 - Questions 4, 8, 9
- Exercise 2.2 Page 84
 - Questions 1, 2, 3, 5
- Exercise 2.3 Page 93
 - Questions 1, 2

Example 1: Maximum element

ALGORITHM *MaxElement*($A[0..n - 1]$)

//Determines the value of the largest element in a given array

//Input: An array $A[0..n - 1]$ of real numbers

//Output: The value of the largest element in A

$maxval \leftarrow A[0]$

for $i \leftarrow 1$ **to** $n - 1$ **do**

if $A[i] > maxval$

$maxval \leftarrow A[i]$

return $maxval$

$$C(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \Theta(n)$$

Example 2: Element uniqueness problem

ALGORITHM *UniqueElements*($A[0..n - 1]$)

//Determines whether all the elements in a given array are distinct

//Input: An array $A[0..n - 1]$

//Output: Returns “true” if all the elements in A are distinct

// and “false” otherwise

for $i \leftarrow 0$ **to** $n - 2$ **do**

for $j \leftarrow i + 1$ **to** $n - 1$ **do**

if $A[i] = A[j]$ **return false**

return true

$$C_{\text{worst}}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^n [(n-1)-(i+1)+1] \in \Theta(n^2)$$

Example 3: Matrix multiplication

ALGORITHM *MatrixMultiplication*($A[0..n-1, 0..n-1]$, $B[0..n-1, 0..n-1]$)

//Multiplies two n -by- n matrices by the definition-based algorithm

//Input: Two n -by- n matrices A and B

//Output: Matrix $C = AB$

for $i \leftarrow 0$ **to** $n - 1$ **do**

for $j \leftarrow 0$ **to** $n - 1$ **do**

$C[i, j] \leftarrow 0.0$

for $k \leftarrow 0$ **to** $n - 1$ **do**

$C[i, j] \leftarrow C[i, j] + A[i, k] * B[k, j]$

return C

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1 = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n = \sum_{i=0}^{n-1} n^2 = n^3 \in \Theta(n^3)$$

$$M(n) = \text{number of multiplications} * \text{size of matrix} = n * n^2$$

Example 4: Gaussian elimination

Algorithm *GaussianElimination*($A[0..n-1,0..n]$)

//Implements Gaussian elimination of an n -by- $(n+1)$ matrix A

for $i \leftarrow 0$ to $n - 2$ do

 for $j \leftarrow i + 1$ to $n - 1$ do

 for $k \leftarrow i$ to n do

$A[j,k] \leftarrow A[j,k] - A[i,k] * A[j,i] / A[i,i]$

Find the efficiency class and a constant factor improvement.

$$M(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} \sum_{k=i}^n 1 = ?$$

Example 5: Counting binary digits

ALGORITHM *Binary*(n)

//Input: A positive decimal integer n

//Output: The number of binary digits in n 's binary representation

$count \leftarrow 1$

while $n > 1$ **do**

$count \leftarrow count + 1$

$n \leftarrow \lfloor n/2 \rfloor$

return $count$

It cannot be investigated the way the previous examples are.

Plan for Analysis of Recursive Algorithms

- Decide on a parameter indicating an input's size.
- Identify the algorithm's basic operation.
- Check whether the number of times the basic op. is executed may vary on different inputs of the same size. (If it may, the worst, average, and best cases must be investigated separately.)
- Set up a recurrence relation with an appropriate initial condition expressing the number of times the basic op. is executed.
- Solve the recurrence (or, at the very least, establish its solution's order of growth) by backward substitutions or another method.

Example 1: Recursive evaluation of $n!$

Definition: $n! = 1 \cdot 2 \cdot \dots \cdot (n-1) \cdot n$ for $n \geq 1$ and $0! = 1$

Recursive definition of $n!$: $F(n) = F(n-1) \cdot n$ for $n \geq 1$ and $F(0) = 1$

ALGORITHM $F(n)$

//Computes $n!$ recursively

//Input: A nonnegative integer n

//Output: The value of $n!$

if $n = 0$ **return** 1

else return $F(n - 1) * n$

Size:

Basic operation:

Recurrence relation:

Solving the recurrence for $M(n)$

$$M(n) = M(n-1) + 1, \quad M(0) = 0$$

Backward substitution:

$$M(n) = M(n-1) + 1, \quad M(0) = 0$$

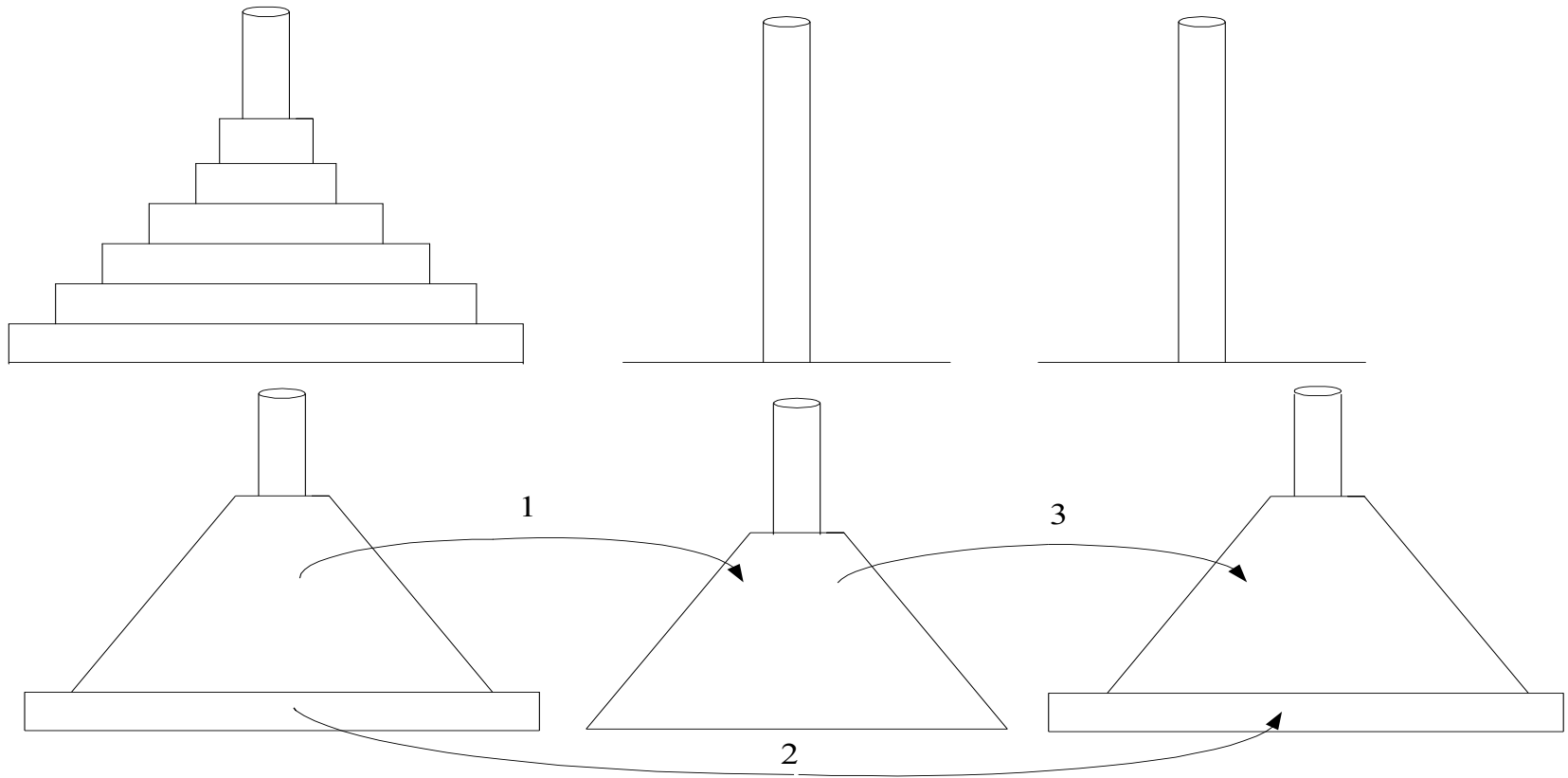
$$= [M(n-2) + 1] + 1 = M(n-2) + 2$$

$$= [M(n-3) + 1] + 2 = M(n-3) + 3$$

...

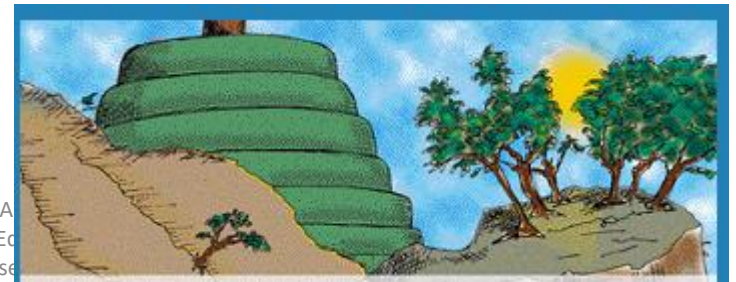
$$M(n) = n?$$

Example 2: The Tower of Hanoi Puzzle



Recurrence for number of moves:

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 2 ©2012 Pearson Education, Inc.
Upper Saddle River, NJ. All Rights Reserved.



Solving recurrence for number of moves

$$M(n) = M(n-1) + 1 + M(n-1),$$

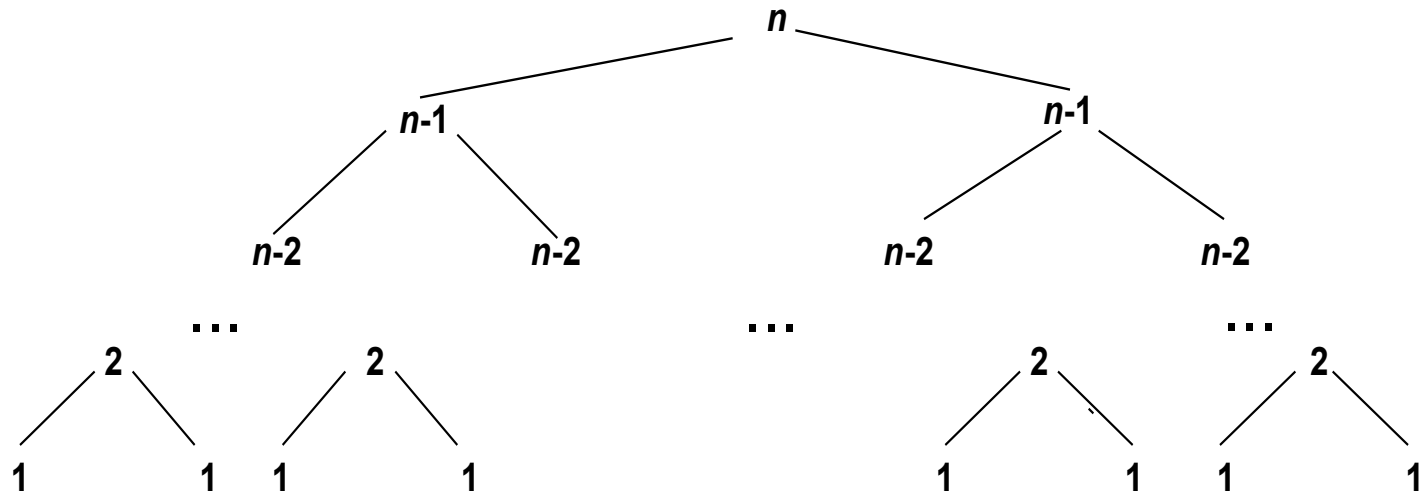
- Move $n-1$ disks from peg 1 to 2
(but not $n-1$ moves)
- One move for n^{th} disk to peg 3
- Move $n-1$ disks from peg 2 to 3

$$M(n) = 2M(n-1) + 1, \quad M(1) = 1$$

Use backward substitution?

$$M(n) = 2^n - 1?$$

Tree of calls for the Tower of Hanoi Puzzle



move() method of Tower of Hanoi?

```
public void move(int n, Stack fromPeg, Stack toPeg,  
                Stack availablePeg){  
    if(n==1){  
        toPeg.push( fromPeg.pop() );  
        return;  
    }  
    move(n-1, fromPeg,    availablePeg, toPeg);  
    move(1,   fromPeg,    toPeg,    availablePeg);  
    move(n-1, availablePeg, toPeg,    fromPeg);  
}
```

Example 3: Counting #bits

ALGORITHM *BinRec*(n)

//Input: A positive decimal integer n

//Output: The number of binary digits in n 's binary representation

if $n = 1$ **return** 1

else return *BinRec*($\lfloor n/2 \rfloor$) + 1

$$A(n) = A(\lfloor n/2 \rfloor) + 1; \quad A(1) = 0$$

$$A(2^k) = A(2^{k-1}) + 1 \quad \text{for } k > 0; \quad A(2^0) = 0$$

Use backward substitute

$$A(2^k) = A(1) + k = k$$

$$\text{if } n = 2^k, \text{ then } k = \log_2 n$$

$$A(n) = \log_2 n$$

Fibonacci numbers

The Fibonacci numbers:

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

The Fibonacci recurrence:

$$F(n) = F(n-1) + F(n-2)$$

$$F(0) = 0$$

$$F(1) = 1$$

General 2nd order linear homogeneous recurrence with constant coefficients:

$$aX(n) + bX(n-1) + cX(n-2) = 0$$

Solving $aX(n) + bX(n-1) + cX(n-2) = 0$

- Set up the characteristic equation (quadratic)

$$ar^2 + br + c = 0$$

- Solve to obtain roots r_1 and r_2

- General solution to the recurrence

if r_1 and r_2 are two distinct real roots: $X(n) = \alpha r_1^n + \beta r_2^n$

if $r_1 = r_2 = r$ are two equal real roots: $X(n) = \alpha r^n + \beta n r^n$

- Particular solution can be found by using initial conditions

Application to the Fibonacci numbers

$$F(n) = F(n-1) + F(n-2) \text{ or } F(n) - F(n-1) - F(n-2) = 0$$

Characteristic equation:

Roots of the characteristic equation:

General solution to the recurrence:

Particular solution for $F(0)=0$, $F(1)=1$:

Computing Fibonacci numbers

1. Definition-based recursive algorithm
2. Nonrecursive definition-based algorithm
3. Explicit formula algorithm
4. Logarithmic algorithm based on formula:

$$\begin{pmatrix} F(n-1) & F(n) \\ F(n) & F(n+1) \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^n$$

for $n \geq 1$, assuming an efficient way of computing matrix powers.

Fibonacci numbers algorithm

Algorithm F(n)

if $n \leq 1$ return n

else return $f(n-1) + f(n-2)$

The numbers of additions need.

$$A(n) = A(n-1) + A(n-2) + 1; \quad A(0) = 0, A(1) = 0$$

Similar to Tower of Hanoi, the efficiency of algorithm is exponential, a^n

Number of Calls for Fibonacci numbers

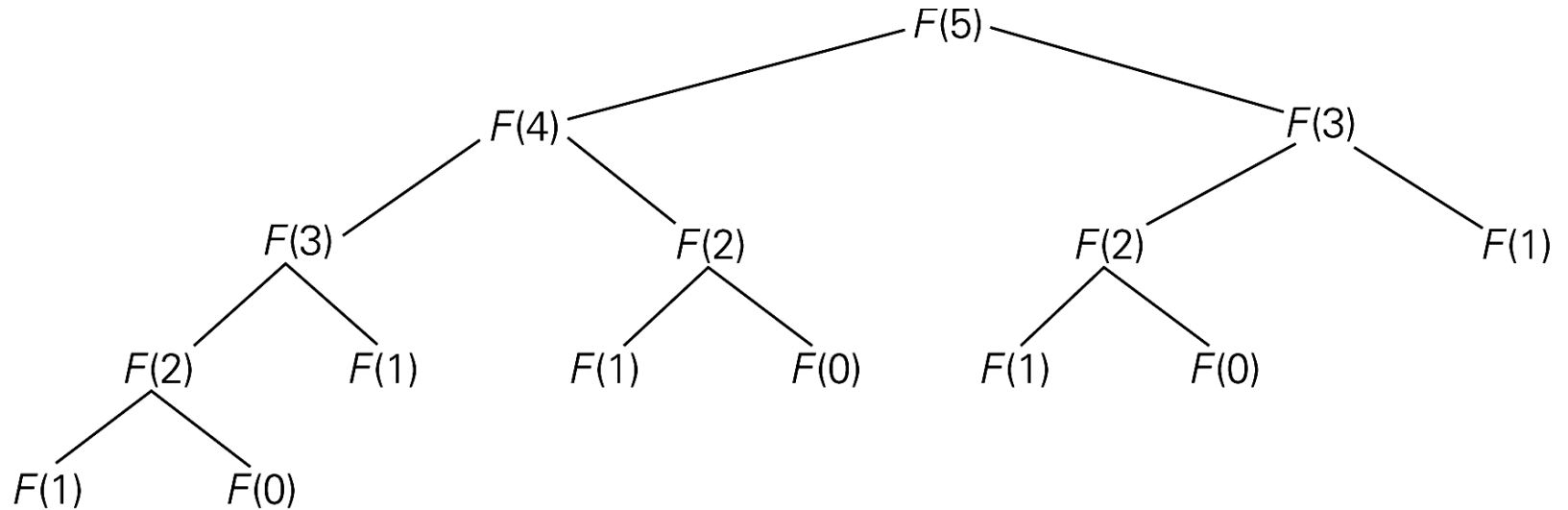
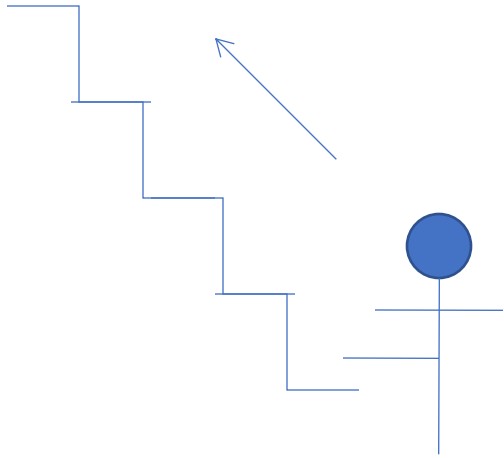


FIGURE 2.6 Tree of recursive calls for computing the 5th Fibonacci number by the definition-based algorithm

Homework

- Demonstrate the moves to solve the Tower of Hanoi problem for 5 dishes. The number of moves must be minimum.
- Exercises 2.4 on page 102
 - Questions: 1, 4, 5
- Exercises 2.5 on page 109
 - Question: 3

Homework



Climbing Stairs: Find the number of different ways to climb an n -stair staircase if each step is either one or two stairs. (For example, a 3-stair staircase can be climbed three ways: 1-1-1, 1-2, and 2-1.)